

LOGIC BUILDING PROGRAMS

Week - 1

1. Write a Python program to check whether a given number is even or odd.

Algorirthm

1. Start
2. Input a number from the user.
3. Compute the remainder when the number is divided by 2 ($\text{num \% 2}$). 4.If the remainder equals 0
4. Print "The given number is Even".
5. Else: Print "The given number is Odd".
6. Stop

```
num = int(input("Enter a number: "))  
  
if num % 2 == 0:  
    print("The given number is Even")  
else:  
    print("The given number is Odd")
```

Enter a number: 6

The number is Even

2. Write a Python program to check whether a number is positive, negative, or zero.

Algorirthm

1. Start
2. Input a number from the user.
3. If the number is greater than 0:
4. Print "The given number is a positive number".
5. Else if the number is equal to 0:
6. Print "The given number is Zero".
7. Else (the number is less than 0):
8. Print "The given number is a negative number".

```
num=int(input("Enter a number: "))  
if num>0:
```

```

    print("The given number is a positive number")
elif num==0:
    print("the given number is Zero")
else:
    print("The given number is a negative number")

```

Enter a number: -5

The given number is a negative number

3. Write a Python program to find the largest among three numbers.

Algorithm

1. Start
2. Input three numbers: num1, num2, num3.
3. If num1 >= num2 and num1 >= num3:
4. Print "num1 is the largest number".
5. Else if num2 >= num1 and num2 >= num3:
6. Print "num2 is the largest number".
7. Else:
8. Print "num3 is the largest number".
9. End

```

num1= int(input("Enter a number: "))
num2= int(input("Enter a number: "))
num3= int(input("Enter a number: "))

if num1>=num2 and num1>=num3:
    print("The largest number is:",num1)

elif num2>=num1 and num2>=num3:
    print("The largest number is:",num2)

else:
    print("The largest number is:",num3)

```

Enter a number: 4

Enter a number: 10

Enter a number: 180

The largest number is: 180

4. Write a Python program to check whether a given number is a prime number.

Algorirthm

1. Start
2. Input a number from the user.
3. If the number is less than or equal to 1: 4. Print "The given number is not a Prime number".
4. Else: For each integer i from 2 to $\sqrt{\text{number}}$:
5. If the number is divisible by i (i.e., $\text{num} \% i == 0$):
6. Print "The given number is not a Prime number".
7. If no divisor is found in the loop:
8. Print "The given number is a Prime number".
9. End

```
# Program to check whether a given number is prime
num = int(input("Enter a number: "))
if num > 1:
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            print("The given number is not a Prime number")
            break
    else:
        print("The given number is a Prime number")
else:
    print("The given number is not a Prime number")
Enter a number: 1
The given number is not a Prime number
```

Week - 2

5. Write a Python program to find the factorial of a number.

Algorirthm

1. Start
2. Input a number num from the user.
3. Initialize a variable factorial = 1.
4. If num < 0:
5. Print "Factorial does not exist for negative numbers".

6. Else:
7. Repeat for i from 1 to num:
8. Multiply factorial = factorial * i.
9. Print "The factorial of num is factorial".
10. End

```
num = int(input("Enter a number: "))

fact = 1
if num < 0:
    print("Factorial does not exist for negative numbers")
else:
    for i in range(1, num + 1):
        fact = fact * i
    print("Factorial of", num, "is", fact)

Enter a number: 5

Factorial of 5 is 120
```

6. Write a Python program to check whether a number is a palindrome.

Algorithm

1. Input a number n.
2. Initialize reversed = 0, original = n.
3. While n > 0: digit = n % 10. reversed = reversed * 10 + digit. n = n // 10.
4. If reversed == original, output "Palindrome"; else "Not Palindrome".
5. End the program

```
n = int(input("Enter a number: "))
original = n
reversed_num = 0
while n > 0:
    digit = n % 10
    reversed_num = reversed_num * 10 + digit
    n = n // 10
print("Palindrome" if reversed_num == original else "Not a
palindrome.")
```

7. Write a Python program to check whether a given string is a palindrome.

Algorithm

1. Start
2. Input a string s from the user.
3. Remove spaces from the string using replace function.

4. Convert the string to lowercase.
5. Reverse the processed string.
6. If the original processed string is equal to its reversed version:
7. Print "The string is a palindrome".
8. Else:
9. Print "The string is not a palindrome".
10. End

```
s = input("Enter a string: ")

# Remove spaces(replace) and convert to lowercase(lower)
s = s.replace(" ", "").lower()

if s == s[::-1]:
    print("The string is a palindrome")
else:
    print("The string is not a palindrome")

Enter a string: LOLLOL
The string is a palindrome
```

Week - 3

8. Write a Python program to print the Fibonacci series up to N terms.

Algorithm

1. Start
2. Input the number of terms N.
3. Initialize two variables:
4. a = 0 (first term)
5. b = 1 (second term)
6. Repeat the following steps for i from 0 to N - 1:
7. Print the value of a.
8. Update values:
9. a = b
10. b = a + b (using old values before update).
11. End

```
# Fibonacci series up to N terms
# Take input from the user
N = int(input("Enter the number of terms: "))
# First two terms
a, b = 0, 1
```

```
print("Fibonacci series up to", N, "terms:")
for i in range(N):
    print(a, end=" ")# Update values
    a, b = b, a + b
```

Enter the number of terms: 7

Fibonacci series up to 7 terms:
0 1 1 2 3 5 8

9. Write a Python program to find the sum of digits of a number.

Algorithm

1. Start
2. Input a number num from the user.
3. Initialize digit_sum = 0.
4. Repeat while num > 0:
5. Extract the last digit using num % 10.
6. Add the digit to digit_sum.
7. Remove the last digit by performing integer division num // 10.
8. After loop ends, print digit_sum.
9. End

```
# Program to find the sum of digits of a number
# Take input from the user
num = int(input("Enter a number: "))
# Initialize sum
digit_sum = 0
# Loop through digits
while num > 0:
    digit_sum += num % 10    # Extract last digit and add
    num //= 10              # Remove last digit
print("Sum of digits:", digit_sum)
```

Enter a number: 12345678910

Sum of digits: 46

10. Write a Python program to count vowels and consonants in a string.

Algorithm

1. Start
2. Input a string text from the user.
3. Define a set of vowels (aeiouAEIOU).
4. Initialize two counters: vowel_count = 0, consonant_count = 0.

5. For each character in text:
6. If the character is an alphabet letter:
7. If the character is in vowels:
8. Increment vowel_count by 1.
9. Else: Increment consonant_count by 1. Print the values of vowel_count and consonant_count.
10. End

```

text = input("Enter a string: ") # Take input from the user
vowels = "aeiouAEIOU" # Define vowels
vowel_count = 0 # Initialize counters
consonant_count = 0
for char in text:
    if char.isalpha(): # Check if character is a letter
        if char in vowels:
            vowel_count += 1
        else:
            consonant_count += 1

print("Number of vowels:", vowel_count)
print("Number of consonants:", consonant_count)

```

Enter a string: Mahek tarannum

Number of vowels: 5
Number of consonants: 8

11. Write a Python program to reverse a string without using built-in functions.

Algorithm: Reverse a String

1. Start
2. Input the string .
3. Initialize an empty string R (to store the reversed result).
4. Find the length of String, say n.
5. Loop from index i = n - 1 down to 0:
6. Append String[i] to R.
7. Output the reversed string R.
8. Stop

```

# Program to reverse a string without using built-in functions
text = input("Enter a string: ")
# Initialize an empty string for the reversed result
reversed_text = ""
for i in range(len(text) - 1, -1, -1):
    reversed_text += text[i]
print("Reversed string:", reversed_text)

```

Enter a string: mom

Reversed string: mom

12. Write a python program to count the occurrence of each character in a string

Algorithm

1. Start
2. Input a string text from the user.
3. Initialize an empty dictionary char_count.
4. For each character in text:
5. If the character already exists in char_count
6. Increment its count by 1.
7. Else: Add the character to char_count with count = 1.
8. After loop ends, print "Character occurrences".
9. For each key-value pair in char_count:
10. Print the character and its count.
11. End

```
# Program to count the occurrence of each character in a string
text = input("Enter a string: ")
char_count = {} # Initialize an empty dictionary
for char in text:
    if char in char_count:
        char_count[char] += 1
    else:
        char_count[char] = 1
print("Character occurrences:")
for char, count in char_count.items():
    print(f"{char}: {count}")
```

Enter a string: hii myself ...

Character occurrences:

```
h: 1
i: 2
 : 2
m: 1
y: 1
s: 1
e: 1
l: 1
f: 1
.: 3
```

13. Write a Python program to create a simple calculator using conditional statements.

Algorithm

1. Start
2. Input num1 and num2
3. Show menu of operations (+, -, *, /)
4. Input user choice
5. If choice = + $\rightarrow$ result = num1 + num2
6. Else if choice = - $\rightarrow$ result = num1 - num2
7. Else if choice = * $\rightarrow$ result = num1 * num2
8. Else if choice = / $\rightarrow$ • If num2 $\neq$ 0 $\rightarrow$ result = num1 / num2 • Else $\rightarrow$ print error
9. Else $\rightarrow$ print "Invalid input"
10. End

```
# Simple calculator using conditional statements

# Take input from the user
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print("Choose operation: +, -, *, /")
operation = input("Enter operation: ")

# Perform calculation using conditional statements
if operation == '+':
    result = num1 + num2
elif operation == '-':
    result = num1 - num2
elif operation == '*':
    result = num1 * num2
elif operation == '/':
    if num2 != 0:
        result = num1 / num2
    else:
        result = "Error! Division by zero."
else:
    result = "Invalid operation."

print("Result:", result)

Enter first number: 1
Enter second number: 8

Choose operation: +, -, *, /
Enter operation: =
```

Result: Invalid operation.

Week-5

14. Write a Python program to implement a menu-driven calculator using a loop (repeat until the user exits).

Algorithm

1. Start: Begin program and enter loop.
2. Show menu: Display options 1–5.
3. Get choice: Read user input for operation.
4. Exit check: If choice = 5, print message and break.
5. Get numbers: Read num1 and num2.
6. Addition: If choice = 1, print num1 + num2.
7. Subtraction/Multiplication: If 2 or 3, print result accordingly.
8. Division: If 4, check num2 $\neq 0 \rightarrow$ print num1 / num2; else print error.
9. Invalid: If none of the above, print "Invalid choice."
10. Repeat: Loop back to menu.

14. *# Menu-driven calculator using a loop*

```
while True:
    print("\n--- Simple Calculator ---")
    print("1. Addition")
    print("2. Subtraction")
    print("3. Multiplication")
    print("4. Division")
    print("5. Exit")

    choice = input("Enter your choice (1-5): ")

    if choice == '5':
        print("Exiting the calculator. Goodbye!")
        break

    # Take two numbers as input
    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))

    if choice == '1':
        print("Result:", num1 + num2)
    elif choice == '2':
        print("Result:", num1 - num2)
    elif choice == '3':
```

```

        print("Result:", num1 * num2)
    elif choice == '4':
        if num2 != 0:
            print("Result:", num1 / num2)
        else:
            print("Error! Division by zero.")
    else:
        print("Invalid choice! Please select a valid option.")

```

--- Simple Calculator ---

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit

Enter your choice (1-5): 5

Exiting the calculator. Goodbye!

15. Write a Python program to generate a multiplication table for a given number (loop until the user stops).

Algorithm

1. Start
2. Repeat indefinitely (loop):
3. Input a number num from the user.
4. Input a limit limit (number of terms).
5. Print "Multiplication Table of num".
6. For each i from 1 to limit:
7. Compute $\text{num} \times i$.
8. Print the result.
9. Ask the user if they want to generate another table.
10. If the user's choice is not "yes":
11. Print "Exiting the program. Goodbye!".
12. Break the loop.
13. End

Program to generate multiplication table until user stops

```

while True:
    num = int(input("Enter a number to generate its multiplication
table: "))
    limit = int(input("Enter the limit (e.g., 10 for up to 10 terms):
"))

```

```

print(f"\nMultiplication Table of {num}:")
for i in range(1, limit + 1):
    print(f"{num} x {i} = {num * i}")

# Ask user if they want to continue
choice = input("\nDo you want to generate another table? (yes/no):")
choice = choice.lower()
if choice != "yes":
    print("Exiting the program. Goodbye!")
    break

```

Enter a number to generate its multiplication table: 9
Enter the limit (e.g., 10 for up to 10 terms): 8

Multiplication Table of 9:

```

9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72

```

Do you want to generate another table? (yes/no):

Exiting the program. Goodbye!

16. Write a Python program to print different patterns using loop concepts (e.g., star patterns, number patterns).

Algorithm

Right Triangle Star Pattern

1. Start
2. Input number of rows rows.
3. For each i from 1 to rows:
4. Print i stars (*).
5. End

Pyramid Star Pattern

6. Start
7. Input number of rows rows.
8. For each i from 1 to rows:
9. Print (rows - i) spaces.
10. Print (2 * i - 1) stars (*).
11. End

Number Triangle Pattern

12. Start
13. Input number of rows rows.
14. For each i from 1 to rows:
15. For each j from 1 to i:
16. Print j followed by a space.
17. Move to the next line.
18. End

```
# Star Pattern (Right Triangle)
rows = int(input("Enter number of rows: "))

print("\nRight Triangle Star Pattern:")
for i in range(1, rows + 1):
    print(" " * i)

# Star Pattern (Pyramid)
rows = int(input("Enter number of rows: "))

print("\nPyramid Star Pattern:")
for i in range(1, rows + 1):
    print(" " * (rows - i) + "*" * (2 * i - 1))

# Number Pattern (Increasing Triangle)
rows = int(input("Enter number of rows: "))

print("\nNumber Triangle Pattern:")
for i in range(1, rows + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

```
Enter number of rows: 4
```

```
Right Triangle Star Pattern:
```

```
*  
**  
***  
****
```

```
Enter number of rows: 3
```

```
Pyramid Star Pattern:
```

```
  *  
 ***  
*****
```

```
Enter number of rows: 2
```

```
Number Triangle Pattern:
```

```
1  
1 2
```

FUNCTION-BASED QUESTIONS

Week - 6

17. Write a Python function that takes a user's name and prints a greeting message.

Algorithm

1. Start
2. Define a function greet_user(name).
3. Inside the function:
4. Print the message "Hello, ! Welcome to Python programming."
5. Call the function with a specific name (e.g., "SAM").
6. End

```
# Function to greet the user  
def greet_user(name):  
    print("Hello,", name + "! Welcome to Python programming.")  
  
# Example usage  
greet_user("SAM")
```

Hello, SAM! Welcome to Python programming.

18. Write a Python function that accepts two numbers and returns their sum.

Algorithm

1. Start
2. Define a function `add_numbers(a, b)` that returns the sum of `a` and `b`.
3. Call the function with two numbers (e.g., 10 and 20).
4. Store the result in a variable `result`.
5. Print the result.
6. End

```
# Function to add two numbers
def add_numbers(a, b):
    return a + b

# Example usage
result = add_numbers(10, 20)
print("The sum is:", result)
```

The sum is: 30

Week – 7

19. Write a Python recursive function to find the factorial of a number.

Algorithm

1. Start
2. Define function `factorial(n)` • If `n == 0` → return 1 (base case). • Else → return `n * factorial(n-1)` (recursive case).
3. Input a number from the user → store in `num`.
4. Call function `factorial(num)`.
5. Store result in a variable (implicitly returned).
6. Print "Factorial of is: ".
7. Stop

```
# Recursive function to find factorial
def factorial(n):
    if n == 0 or n == 1:    # Base case
        return 1
    else:
        return n * factorial(n - 1) # Recursive call
```

```
# Example usage
num = 5
print("Factorial of", num, "is:", factorial(num))

Factorial of 5 is: 120
```

20. Write a Python lambda function to check whether a number is even.

Algorithm to check whether a number is even using a lambda function:

1. Start
2. Define a lambda function `is_even = lambda x: x % 2 == 0`.
3. Input a number `num` from the user.
4. Pass `num` to the lambda function `is_even(num)`.
5. If the result is `True`:
6. Print "num is even".
7. Else:
8. Print "num is odd".
9. Stop

```
# Lambda function to check even number
is_even = lambda x: x % 2 == 0

# Example usage
num = int(input("Enter a number: "))
if is_even(num):
    print(num, "is even")
else:
    print(num, "is odd")
```

21. Write a Python program to calculate factorial using recursion with input validation.

Algorithm to calculate factorial using recursion with input validation:

1. Start
2. Define `factorial(n) → base case (0 or 1 → 1), else n * factorial(n-1)`.
3. Loop: ask user for input (q to quit).
4. If input = q → exit.
5. If input invalid or negative → show error.
6. Else → compute factorial and print result.
7. Repeat until quit.
8. Stop

```
def factorial(n):
    """Recursive function to calculate factorial of n."""
    if n == 0 or n == 1:
        return 1
```

```

    else:
        return n * factorial(n - 1)

def main():
    try:
        # Take input from user
        num = int(input("Enter a non-negative integer: "))

        # Validate input
        if num < 0:
            print("Error: Factorial is not defined for negative numbers.")
        else:
            print(f"Factorial of {num} is {factorial(num)}")

    except ValueError:
        print("Error: Please enter a valid integer.")

if __name__ == "__main__":
    main()

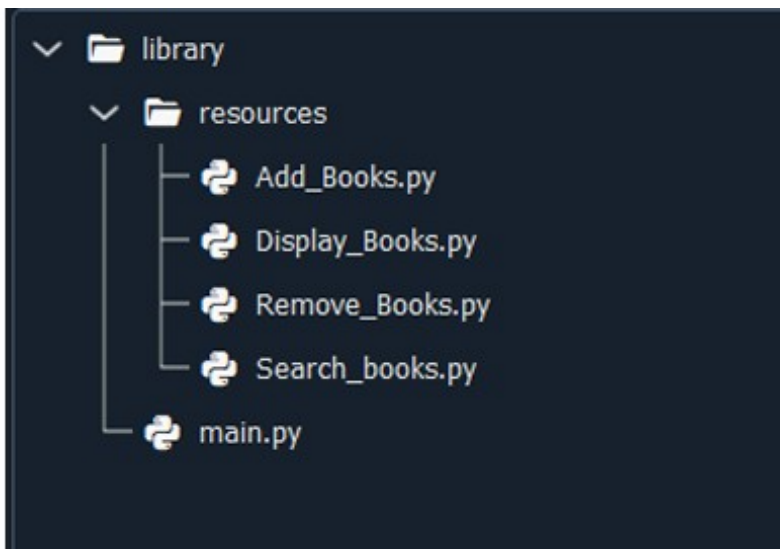
```

Enter a non-negative integer: 2

Factorial of 2 is 2

Week-8

22. Write a Python program to create a Library Book Management System using functions.



```

library = []
def add_book(title, author, year, isbn):

```

```

for book in library:
    if book['isbn'] == isbn:
        print("Book with this ISBN already exists.")
        return
library.append({
    'title': title,
    'author': author,
    'year': year,
    'isbn': isbn
})
print("Book added successfully.")
def remove_book(isbn):
    global library
    for book in library:
        if book['isbn'] == isbn:
            library.remove(book)
            print("Book removed.")
            return
    print("Book not found.")
def search_books(field, value):
    results = []
    for book in library:
        if str(book.get(field, '')).lower() == str(value).lower():
            results.append(book)
    return results
def display_books():
    if not library:
        print("Library is empty.")
        return
    for idx, book in enumerate(library, 1):
        print(f"{idx}. {book['title']} by {book['author']} ({book['year']}) [ISBN: {book['isbn']}]")
# Main program loop
def main():
    while True:
        print("\nLibrary System")
        print("1. Add Book")
        print("2. Remove Book")
        print("3. Search Book")
        print("4. Display All Books")
        print("5. Exit")
        choice = input("Enter choice: ")
        if choice == '1':
            title = input("Enter title: ")
            author = input("Enter author: ")
            year = input("Enter year: ")
            isbn = input("Enter ISBN: ")
            add_book(title, author, year, isbn)
        elif choice == '2':

```

```

        isbn = input("Enter ISBN to remove: ")
        remove_book(isbn)
    elif choice == '3':
        field = input("Search by (title/author/year): ").lower()
        value = input("Enter search value: ")
        results = search_books(field, value)
        if results:
            for book in results:
                print(f"{book['title']} by {book['author']} ({book['year']}) [ISBN: {book['isbn']}]")
            else:
                print("No matching books found.")
        elif choice == '4':
            display_books()
        elif choice == '5':
            print("Exiting...")
            break
        else:
            print("Invalid choice.")
# Run the main() function only if this script is being run directly, not if it is imported into another Python file
if __name__ == "__main__":
    main()

```

Library System

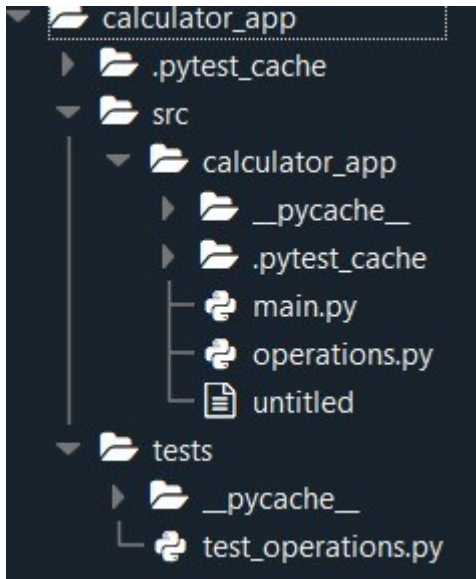
1. Add Book
2. Remove Book
3. Search Book
4. Display All Books
5. Exit

Enter choice: 5

Exiting...

Week - 9

23. Write a Python project to build a Calculator using modular programming (separate module for operations).



Week - 10

```
def add(a: float, b: float) -> float:
    return a + b
def subtract(a: float, b: float) -> float:
    return a - b
def multiply(a: float, b: float) -> float:
    return a * b
def divide(a: float, b: float) -> float:
    if b == 0:
        raise ValueError("Cannot divide by zero.")
    return a / b
import pytest
from src.calculator_app.operations import add, subtract, multiply,
divide

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0
    assert add(0, 0) == 0

def test_subtract():
    assert subtract(5, 3) == 2
    assert subtract(0, 5) == -5

def test_multiply():
    assert multiply(2, 4) == 8
```

```

    assert multiply(-1, 5) == -5
    assert multiply(0, 100) == 0

def test_divide():
    assert divide(10, 2) == 5
    assert divide(9, 3) == 3

def test_divide_by_zero():
    with pytest.raises(ValueError):
        divide(5, 0)

import os
import pytest
from src.calculator_app.operations import add, subtract, multiply,
divide

def calculator():
    print("Welcome to Mini Calculator!\n")
    while True:
        print("\nChoose operation:")
        print("1: Add")
        print("2: Subtract")
        print("3: Multiply")
        print("4: Divide")
        print("5: Exit")

        choice = input("Enter choice (1/2/3/4/5): ")

        if choice == "5":
            print("Exiting calculator...\n")
            break

        try:
            a = float(input("Enter first number: "))
            b = float(input("Enter second number: "))
        except ValueError:
            print("Please enter valid numbers!\n")
            continue

        if choice == "1":
            print(f"Result: {add(a, b)}\n")
        elif choice == "2":
            print(f"Result: {subtract(a, b)}\n")
        elif choice == "3":
            print(f"Result: {multiply(a, b)}\n")
        elif choice == "4":
            try:
                print(f"Result: {divide(a, b)}\n")
            except ValueError as e:
                print(f"Error: {e}\n")

```

```

        else:
            print("Invalid choice! Please try again.\n")

def run_tests():
    print("Running automated tests...")
    # Absolute path to test_operations.py
    project_root =
os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
    test_file_path = os.path.join(project_root, "tests",
    "test_operations.py")

    # Run pytest programmatically
    result = pytest.main([test_file_path, "-q", "--tb=short"])
    if result == 0:
        print("All tests passed! ☐")
    else:
        print("Some tests failed! ☐")

if __name__ == "__main__":
    calculator()
    run_tests()

import os
import pytest
from src.calculator_app.operations import add, subtract, multiply,
divide

def calculator():
    print("Welcome to Mini Calculator!\n")
    while True:
        print("\nChoose operation:")
        print("1: Add")
        print("2: Subtract")
        print("3: Multiply")
        print("4: Divide")
        print("5: Exit")

        choice = input("Enter choice (1/2/3/4/5): ")

        if choice == "5":
            print("Exiting calculator...\n")
            break

        try:
            a = float(input("Enter first number: "))
            b = float(input("Enter second number: "))
        except ValueError:
            print("Please enter valid numbers!\n")
            continue

```

```

    if choice == "1":
        print(f"Result: {add(a, b)}\n")
    elif choice == "2":
        print(f"Result: {subtract(a, b)}\n")
    elif choice == "3":
        print(f"Result: {multiply(a, b)}\n")
    elif choice == "4":
        try:
            print(f"Result: {divide(a, b)}\n")
        except ValueError as e:
            print(f"Error: {e}\n")
    else:
        print("Invalid choice! Please try again.\n")

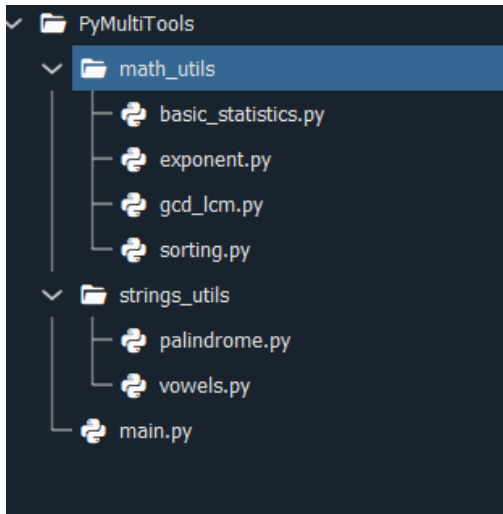
def run_tests():
    print("Running automated tests...")
    # Absolute path to test_operations.py
    project_root =
os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
    test_file_path = os.path.join(project_root, "tests",
"test_operations.py")

    # Run pytest programmatically
    result = pytest.main([test_file_path, "-q", "--tb=short"])
    if result == 0:
        print("All tests passed! 🎉")
    else:
        print("Some tests failed! 🚫")

if __name__ == "__main__":
    calculator()
    run_tests()

```

24. Write a Python program that applies modular programming principles and defines multiple reusable functions.



```
## basic_statistics
import statistics

def mean(numbers):
    """Return the mean of the list."""
    return statistics.mean(numbers)

def median(numbers):
    """Return the median of the list."""
    return statistics.median(numbers)

def mode(numbers):
    """Return the mode of the list."""
    return statistics.mode(numbers)

#exponent
def exponent(base, power):
    """Return base raised to the power."""
    return base ** power

# gcd_lcm.py
import math

def gcd(a, b):
    """Return the greatest common divisor of a and b."""
    return math.gcd(a, b)

def lcm(a, b):
```

```

    """Return the least common multiple of a and b."""
    return abs(a * b) // math.gcd(a, b)

# sorting.py
def sort_ascending(numbers):
    """Return numbers sorted in ascending order."""
    return sorted(numbers)

def sort_descending(numbers):
    """Return numbers sorted in descending order."""
    return sorted(numbers, reverse=True)

#palindrome
def is_palindrome(text):
    """Check if a string is a palindrome."""
    cleaned = ''.join(ch.lower() for ch in text if ch.isalnum())
    return cleaned == cleaned[::-1]

# vowels.py
def is_vowel(char):
    """Check if a character is a vowel."""
    return char.lower() in ['a', 'e', 'i', 'o', 'u']

def extract_vowels(sentence):
    """Return a list of vowels found in the sentence."""
    return [ch for ch in sentence if is_vowel(ch)]

def count_vowels(sentence):
    """Return the number of vowels in the sentence."""
    return len(extract_vowels(sentence))

# main.py
from math_utils import sorting, exponent, basic_statistics, gcd_lcm
from strings_utils import palindrome, vowels

def main():
    print("Choose option:")
    print("1. Sorting Numbers")
    print("2. Exponential Calculation")
    print("3. Basic Statistics")
    print("4. GCD and LCM")
    print("5. Palindrome Check")
    print("6. Vowel Analysis")

```

```

choice = input("Enter choice: ")

if choice == '1':
    nums = list(map(int, input("Enter numbers separated by space: ").split()))
    print("Ascending:", sorting.sort_ascending(nums))
    print("Descending:", sorting.sort_descending(nums))

elif choice == '2':
    base = int(input("Enter base: "))
    power = int(input("Enter power: "))
    print("Result:", exponent.exponent(base, power))

elif choice == '3':
    nums = list(map(int, input("Enter numbers separated by space: ").split()))
    print("Mean:", basic_statistics.mean(nums))
    print("Median:", basic_statistics.median(nums))
    try:
        print("Mode:", basic_statistics.mode(nums))
    except Exception:
        print("Mode: No unique mode found")

elif choice == '4':
    a, b = map(int, input("Enter two numbers: ").split())
    print("GCD:", gcd_lcm.gcd(a, b))
    print("LCM:", gcd_lcm.lcm(a, b))

elif choice == '5':
    text = input("Enter text: ")
    print("Is Palindrome?", palindrome.is_palindrome(text))

elif choice == '6':
    sentence = input("Enter sentence: ")
    print("Vowels:", vowels.extract_vowels(sentence))
    print("Vowel Count:", vowels.count_vowels(sentence))

else:
    print("Invalid choice")

main()

```

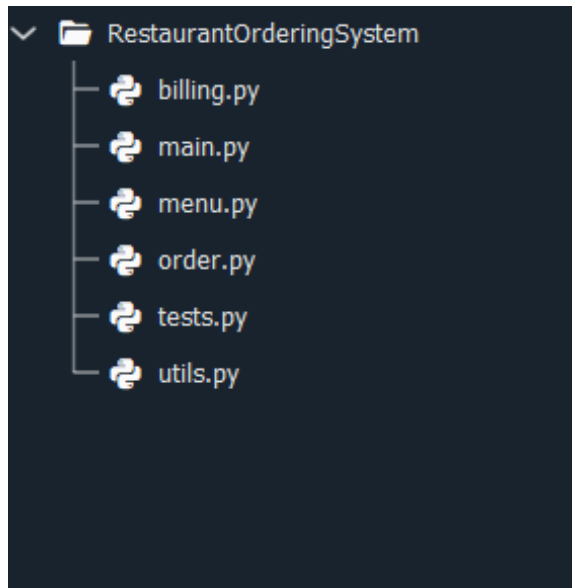
Week - 11

25. Write a Python program using modular programming principles and demonstrate:

Input validation

Testing (minimum 3 test cases)

Debugging practice with comments



```
# menu.py
def get_menu():
    """Return restaurant menu as dictionary."""
    return {
        "Pizza": 250,
        "Burger": 150,
        "Pasta": 200,
        "Salad": 120,
        "Coffee": 80
    }
```

```
# order.py
def get_menu():
    """Return restaurant menu as dictionary."""
    return {
        "Pizza": 250,
        "Burger": 150,
        "Pasta": 200,
        "Salad": 120,
        "Coffee": 80
    }
```

```
}
```

```
# billing.py
```

```
from menu import get_menu
```

```
def calculate_bill(order):  
    """Calculate subtotal, tax, and total bill."""  
    menu = get_menu()  
    subtotal = sum(menu[item] * qty for item, qty in order.items())  
    tax = subtotal * 0.05 # 5% tax  
    total = subtotal + tax  
    return {"subtotal": subtotal, "tax": tax, "total": total}
```

```
# tests.py
```

```
import pytest
```

```
from order import create_order
```

```
def test_valid_order():  
    order = create_order({"Pizza": 2, "Coffee": 1})  
    assert order == {"Pizza": 2, "Coffee": 1}
```

```
def test_invalid_item():  
    with pytest.raises(ValueError) as e:  
        create_order({"Sandwich": 1})  
    assert "Invalid item" in str(e.value)
```

```
def test_negative_quantity():  
    with pytest.raises(ValueError) as e:  
        create_order({"Pizza": -2})  
    assert "Invalid quantity" in str(e.value)
```

```
def test_zero_quantity():  
    with pytest.raises(ValueError) as e:  
        create_order({"Burger": 0})  
    assert "Invalid quantity" in str(e.value)
```

```
def test_string_quantity():  
    with pytest.raises(ValueError) as e:  
        create_order({"Pasta": "two"})  
    assert "Invalid quantity" in str(e.value)
```

```
def test_float_quantity():  
    with pytest.raises(ValueError) as e:  
        create_order({"Salad": 2.5})  
    assert "Invalid quantity" in str(e.value)
```

```
def test_empty_order():  
    with pytest.raises(ValueError) as e:  
        create_order({})
```

```

        assert "Order cannot be empty" in str(e.value)

# utils.py
import logging

# Configure logging for debugging
logging.basicConfig(level=logging.DEBUG, format="%(levelname)s: %(message)s")

def validate_item(item, menu):
    """Check if the item exists in the menu."""
    if item not in menu:
        logging.error(f"Invalid item attempted: {item}")
        return False
    logging.debug(f"Item validated: {item}")
    return True

def validate_quantity(qty):
    """Check if quantity is valid (integer > 0)."""
    if not isinstance(qty, int):
        logging.error(f"Quantity not integer: {qty}")
        return False
    if qty <= 0:
        logging.error(f"Quantity out of range: {qty}")
        return False
    logging.debug(f"Quantity validated: {qty}")
    return True

def validate_order(order, menu):
    """Validate the entire order dictionary."""
    if not order:
        raise ValueError("Order cannot be empty")

    for item, qty in order.items():
        if not validate_item(item, menu):
            raise ValueError(f"Invalid item: {item}")
        if not validate_quantity(qty):
            raise ValueError(f"Invalid quantity for {item}: {qty}")

    return True

# main.py
from menu import get_menu
from billing import calculate_bill
from order import create_order

def main():
    print("Welcome to Mahek's Restaurant 🍽️")

```

```

menu = get_menu()

# Display menu
print("\nMenu:")
for i, (item, price) in enumerate(menu.items(), start=1):
    print(f"{i}. {item} - ₹{price}")

try:
    # Ask user for multiple choices
    choices = input("\nEnter the numbers of your choices (comma
separated): ").strip()
    if not choices:
        raise ValueError("You must select at least one item.")

    choice_list = choices.split(",")
    items = {}

    for choice in choice_list:
        choice = choice.strip()
        if not choice.isdigit():
            raise ValueError("Choice must be a number.")
        choice = int(choice)

        if choice < 1 or choice > len(menu):
            raise ValueError("Choice out of range.")

        selected_item = list(menu.keys())[choice - 1]
        qty = input(f"Enter quantity for {selected_item}:
").strip()
        if not qty.isdigit():
            raise ValueError("Quantity must be a number.")
        qty = int(qty)
        if qty <= 0:
            raise ValueError("Quantity must be greater than 0.")

        items[selected_item] = qty

    # Validate and calculate bill
    order = create_order(items)
    bill = calculate_bill(order)

    print("\nYour Order:", order)
    print("Bill:", bill)

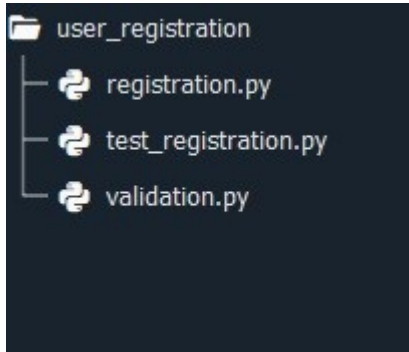
except ValueError as e:
    print("Error:", e)

if __name__ == "__main__":
    main()

```

Week-12

26. Write a Python project for a User Registration System with input validation, testing, and debugging documentation.



```
#registration.py
import validation

class UserRegistrationSystem:
    def __init__(self):
        self.users = {}

    def register_user(self, username, email, password):
        # Validate inputs
        if not validation.validate_username(username):
            return "Invalid username. Must be at least 3 characters."
        if not validation.validate_email(email):
            return "Invalid email format."
        if not validation.validate_password(password):
            return "Password must be at least 8 characters, include a
number and a special character."

        # Check if user already exists
        if username in self.users:
            return "Username already exists."

        # Save user
        self.users[username] = {"email": email, "password": password}
        return "User registered successfully!"

#validation.py
import re

def validate_username(username):
    return len(username) >= 3

def validate_email(email):
    pattern = r'^[\w\.-]+@[\w\.-]+\.\w+$'
    return re.match(pattern, email) is not None
```

```

def validate_password(password):
    # At least 8 chars, one digit, one special char
    if len(password) < 8:
        return False
    if not re.search(r'\d', password):
        return False
    if not re.search(r'[!@#$%^&*(),.?":{}|<>]', password):
        return False
    return True

#test_registration.py
import unittest
from registration import UserRegistrationSystem

class TestUserRegistrationSystem(unittest.TestCase):
    def setUp(self):
        self.system = UserRegistrationSystem()

    def test_valid_registration(self):
        result = self.system.register_user("john123",
        "john@example.com", "Pass@123")
        self.assertEqual(result, "User registered successfully!")

    def test_invalid_username(self):
        result = self.system.register_user("jo", "jo@example.com",
        "Pass@123")
        self.assertEqual(result, "Invalid username. Must be at least 3
characters.")

    def test_invalid_email(self):
        result = self.system.register_user("john123",
        "johnexample.com", "Pass@123")
        self.assertEqual(result, "Invalid email format.")

    def test_invalid_password(self):
        result = self.system.register_user("john123",
        "john@example.com", "password")
        self.assertEqual(result, "Password must be at least 8
characters, include a number and a special character.")

    def test_duplicate_username(self):
        self.system.register_user("john123", "john@example.com",
        "Pass@123")
        result = self.system.register_user("john123",
        "john2@example.com", "Pass@456")
        self.assertEqual(result, "Username already exists.")

if __name__ == "__main__":
    unittest.main()

```

Week - 13

27. Write a mini-project in Python incorporating various programming concepts (loops, functions, lists, modules, validation, testing).

```
#backend/app_client
import os
from dotenv import load_dotenv
from groq import Groq

# Load environment variables
load_dotenv()

API_KEY = os.getenv("GROQ_API_KEY")

if not API_KEY:
    raise ValueError("GROQ_API_KEY not found in .env file")

# Create Groq client
client = Groq(api_key=API_KEY)

def get_ai_response(user_message: str) -> str:
    if not user_message.strip():
        return "Say something ☐"

    try:
        response = client.chat.completions.create(
            model="openai/gpt-oss-120b",
            messages=[
                {
                    "role": "system",
                    "content": (
                        "You are Nova AI, a smart, concise, and\nhelpful assistant. "\n"Answer clearly and practically. Avoid\nunnecessary verbosity."
                    )
                },
                {
                    "role": "user",
                    "content": user_message
                }
            ],
            temperature=0.6,
            max_tokens=512
        )
```

```

        return response.choices[0].message.content

    except Exception as e:
        return f"Error talking to AI: {str(e)}"
#backend/app.py
from flask import Flask, request, jsonify
from flask_cors import CORS

from ai_client import get_ai_response

app = Flask(__name__)
CORS(app) # allows frontend (browser) to talk to backend

@app.route("/chat", methods=["POST"])
def chat():
    """
    Receives user message from frontend,
    sends it to AI, and returns AI response.
    """

    data = request.get_json()

    if not data or "message" not in data:
        return jsonify({"error": "No message provided"}), 400

    user_message = data["message"]

    ai_reply = get_ai_response(user_message)

    return jsonify({"reply": ai_reply})

if __name__ == "__main__":
    app.run(debug=True)
#backend/.env
GROQ_API_KEY=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
#backend/requirements.txt
flask
flask-cors
python-dotenv
requests
groq
#frontend/index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Nova AI</title>
  <link rel="stylesheet" href="style.css" />

```

```

</head>
<body>

  <div class="chat-container">
    <h1>☺ Nova AI</h1>

    <div id="chat-box"></div>

    <div class="input-area">
      <input type="text" id="user-input" placeholder="Ask Nova something..." />
      <button onclick="sendMessage()">Send</button>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>

```

#frontend/style.css

```

body {
  background-color: #0f0f0f;
  font-family: Arial, sans-serif;
  display: flex;
  justify-content: center;
  margin-top: 50px;
  color: white;
}

```

```

.chat-container {
  width: 400px;
  background: #1a1a1a;
  padding: 20px;
  border-radius: 10px;
}

```

```

#chat-box {
  height: 300px;
  overflow-y: auto;
  background: #111;
  padding: 10px;
  margin-bottom: 10px;
  border-radius: 5px;
}

```

```

.message {
  margin: 8px 0;
}

```

```

.user {

```

```

    color: #00bcd4;
}

.ai {
    color: #90ee90;
}

.input-area {
    display: flex;
}

input {
    flex: 1;
    padding: 8px;
    border: none;
    border-radius: 5px;
}

button {
    margin-left: 5px;
    padding: 8px 12px;
    border: none;
    border-radius: 5px;
    background: #00bcd4;
    cursor: pointer;
}

```

#frontend/script.js

```

async function sendMessage() {
    const input = document.getElementById("user-input");
    const chatBox = document.getElementById("chat-box");

    const message = input.value.trim();
    if (!message) return;

    // Show user message
    chatBox.innerHTML += `<div class="message user">You: ${
message}</div>`;
    input.value = "";

    try {
        const response = await fetch("http://127.0.0.1:5000/chat", {
            method: "POST",
            headers: {
                "Content-Type": "application/json"
            },
            body: JSON.stringify({ message })
        });

        const data = await response.json();
    }

```

```
        chatBox.innerHTML += `<div class="message ai">Nova: ${data.reply}</div>`;
        chatBox.scrollTop = chatBox.scrollHeight;

    } catch (error) {
        chatBox.innerHTML += `<div class="message ai">Nova: Error connecting to backend</div>`;
    }
}
```